Does momentum have a measurable effect on the next point in a tennis match?

COMP90051 Statistical Machine Learning — Group 51
Benjamin Morrison (ID), Dev Karan Singh (ID), Faisal Abdullah Khan (ID)

1 Introduction

The obvious prediction task on point-by-point tennis data is "who wins the next point". That task is trivially confounded and not our research question. Service alternates game by game, and the stronger player wins more points, so apparent "streaks" arise even when every point is independent: "won the last three points" frequently means only "is serving this game" or "is the better player". We therefore ask a falsifiable question about *temporal structure*:

> *Does the* order *of the preceding points carry information about the winner of the next point,* beyond *what is explained by the server, the score and pressure context, and player strength — and if so, how many points back does that memory extend?*

This cannot be answered by a feature-importance table; it requires an experimental design with controls. We use three experiments: a **negative (shuffle) control** that destroys point order while preserving serve-conditional base rates, a **memory-horizon** sweep that measures predictive gain as a function of how much history each model sees, and a **positive control** on simulated data with a planted momentum effect of known size, which calibrates the smallest effect our pipeline could detect. The negative result is as informative as the positive one: if models do no better on real data than on the order-destroyed null, momentum carries no predictive signal.

Dataset. Jeff Sackmann's Grand Slam point-by-point collection, covering most main-draw singles matches at the four majors since 2011 (`https://github.com/JeffSackmann/tennis_slam_pointbypoint`). Each row is one point, with the server, the point winner, set/game/point indices, running score, and — where available — rally length, serve speed, and ace/winner/unforced-error flags. After cleaning we retain N points across M matches.

2 Literature review

Dataset. The Sackmann collection [1] is scraped from the official Grand Slam sites; field availability varies substantially by tournament and year (serve speed, serve number and rally length are missing for many matches), which shapes our feature choices.

Is tennis memoryless? Klaassen and Magnus [2] fit a dynamic binary panel model to Wimbledon points and find that points are *not* perfectly independent and identically distributed, but that deviations are small; winning the previous point has a minor positive association with winning the next. This frames momentum as a small-effect question where confound control is decisive, and motivates our shuffle control.

The hot hand. The "hot hand" debate originates with Gilovich, Vallone and Tversky [3], who argued the belief in streaks in basketball was largely a cognitive misperception of randomness. Miller and Sanjurjo [4] later showed that the standard streak estimator is itself biased in finite samples, so naïve "win-after-a-streak" statistics overstate reversals. Our within-match shuffle inherits the same bias, so comparing real against shuffled performance cancels it rather than being misled by it.

Algorithm paper (not covered in class). Our complex model is the Relational Recurrent Neural Network of Santoro et al. [5] (NeurIPS 2018). A standard LSTM — which the subject does cover — compresses the past into a single hidden vector. The Relational Memory Core instead maintains a *matrix* of memory slots and, at every step, lets those slots interact through multi-head self-attention before gating them into the next state. This slot-to-slot self-attention is the component that distinguishes it from the recurrent architectures taught in COMP90051 and is a post-2017 development outside the syllabus; we use it to ask whether a model that can relate separated past events recovers momentum structure the hand-crafted features miss.

3 Methods

3.1 Preprocessing and targets

We parse every `*-points.csv`, normalise heterogeneous column names to a canonical schema at load time, and drop points with no valid winner (data gaps). Points are ordered within each match by set/game/point index. We define two targets: **p1_won** (did player 1 win the point) and the serve-adjusted **server_won**; momentum is tested as information beyond the latter. Crucially, all history-based features for a point use *only* points strictly earlier in the same match, so no label leaks into its own predictors, and history never crosses match boundaries.

3.2 Feature construction

Simple preprocessing alone is disallowed and, more importantly, insufficient to capture order. From the raw point stream we construct: rolling win rates over the last $w \in \{1, 3, 5, 10, 20\}$ points; exponentially-weighted win rates at several decay rates $d \in \{0.5, 0.7, 0.85, 0.95\}$ (effective memory $\approx 1/(1-d)$, giving short- to long-memory views); a signed current-streak length; pressure indicators derived from the score (break point, set point, deciding set, game/set differential); and, as controls, the server indicator and a rolling player-strength proxy. The key non-obvious feature is **surprise**: an exponentially-weighted average of (outcome − expected win probability), where the expectation comes from a server-and-score-only baseline fitted on the training fold. Surprise measures momentum as a *departure from what serve and score already predict*, separating

genuine sequential effect from the mechanical advantage of serving. Window $\times$ server and decay $\times$ pressure interactions expand the design matrix past the required feature count.

3.3 Three algorithms of increasing complexity

We compare models spanning the model-complexity spectrum, all evaluated on identical cross-validation folds for fairness.

- **Simple — ℓ_2 logistic regression**, implemented from scratch with batch gradient descent. Linear in the engineered features; cannot represent order beyond what the features encode. Tuned hyperparameter: regularisation strength λ.
- **Medium — gradient-boosted trees** on the same features, capturing non-linear feature interactions (e.g. streaks that matter only under pressure). Tuned hyperparameter: number of boosting rounds.
- **Complex — Relational RNN** [5], which reads the *raw sequence* of recent points rather than the engineered features, and so can in principle discover sequential structure the features omit. Tuned hyperparameter: number of memory slots.

The simple and medium models share inputs, so any difference between them is attributable to model class; the complex model changes the input representation, so comparing it against the others isolates the value of learning from the raw sequence.

3.4 Cross-validation (from scratch)

Because the data is temporal, evaluation must predict the future from the past; random point-level splitting would leak the very sequential signal under study. We implement, without third-party libraries, a temporal match-level scheme: the time-ordered matches are cut into 11 contiguous blocks and repetition r trains on block r and tests on block $r{+}1$, giving 10 repetitions. Splits are at the match level (an entire match is train or test, never both). The same splits are used for all three models, and the 10 repetitions supply the error bars reported throughout.

3.5 Nested cross-validation and tuning (from scratch)

Hyperparameters are selected by an inner cross-validation nested inside each outer training fold, implemented from scratch: within the outer training block we run $k{=}3$ match-level folds, score each candidate value by inner log-loss, refit the best on the full outer-training block, and evaluate once on the untouched outer-test block. Each grid has ≥ 3 values and is centred so the middle value should win most often; we report the selection frequencies and widen any grid whose extreme value dominates.

3.6 Evaluation metrics (from scratch)

We compute log-loss, Brier score, accuracy, AUC (via the rank identity), and calibration curves ourselves; third-party metric functions are not used. Log-loss and Brier are the primary metrics because the question concerns probability quality, not hard classification. Learning curves are reported against training-set size (not against epochs, which the brief excludes).

4 Results and discussion

All numeric cells below are placeholders (in red) to be replaced by the outputs in `results/*.json` *after running the three experiment scripts; the figures are produced by* `report/make_figures.py`*. The surrounding analysis is written to hold for either outcome.*

4.1 R1 — Shuffle control

Table 1 compares each model on real data against the order-destroyed null. The momentum signal is the real-minus-shuffled gap; if it is within the error bars, order carries no detectable predictive information.

Table 1: Predictive performance on real vs. order-shuffled data (mean $\pm$ s.e. over 10 temporal folds). Lower log-loss/Brier is better; higher AUC is better.

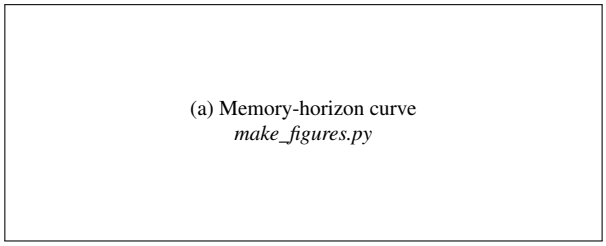
Model	Data	Log-loss ↓	Brier ↓	AUC ↑
Logistic reg.	real	0.000	0.000	0.000
	shuffled	0.000	0.000	0.000
Grad. boosting	real	0.000	0.000	0.000
	shuffled	0.000	0.000	0.000
Relational RNN	real	0.000	0.000	0.000
	shuffled	0.000	0.000	0.000

4.2 R2 — Memory horizon

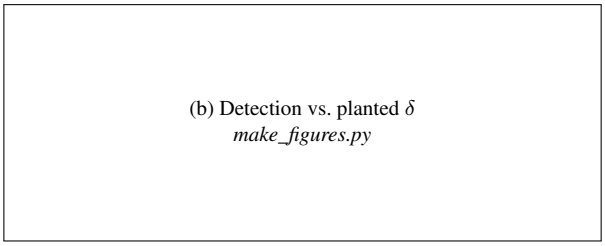
Figure 1(a) plots predictive gain over the zero-history baseline as history length grows. The point at which the curve flattens is the *momentum horizon*; a curve that never rises above its error band is evidence of no exploitable memory.

4.3 R3 — Positive control

Figure 1(b) shows detection (AUC above the $\delta{=}0$ band) as a function of the planted momentum size δ on simulated matches. The smallest δ the pipeline reliably detects is its sensitivity floor; reporting the real-data effect against this floor is what makes a null result meaningful rather than an artefact of weak methods.



(a) Memory horizon



(b) Positive control

Figure 1: (a) Predictive gain vs. history length with error bars; the plateau is the momentum horizon. (b) Detection vs. planted effect size on simulated data, establishing the sensitivity floor.

4.4 Discussion

Why each model behaves as it does. Logistic regression can only express momentum that is already encoded in the engineered windows and the surprise feature; it is the right floor because any gain it shows is directly interpretable as "these summaries of the past help". Gradient boosting adds value only if momentum is *non-linear or conditional* — for instance if streaks matter under break points but not otherwise — so a boosting gain over logistic regression localises the effect to interactions rather than to the raw recency summaries. The Relational RNN can represent order the features never materialise, but the momentum signal (per Klaassen and Magnus) is small and the target is high-variance, so its large capacity is as likely to overfit as to help; the honest expectation is that it does *not* comfortably beat the simpler models, and if it does not, that is itself evidence that hand-crafted recency features already capture whatever short-range memory exists.

Reading the controls together. The two controls bracket the real result. The shuffle control guards against reporting a confound as momentum: because the shuffle preserves serve-conditional base rates, any real-minus-shuffled gap cannot be explained by serving or by overall player strength. The positive control guards against the opposite error — declaring "no momentum" when the method simply lacks power — by exhibiting the smallest planted effect the same pipeline recovers. A credible conclusion requires both: a real gap that clears the shuffle band *and* exceeds the sensitivity floor.

Advantages and disadvantages. The engineered-feature models are interpretable and cheap, and their coefficients/importances say *which* horizon of history matters, but they can only find momentum we thought to encode. The Relational RNN needs no feature guess and can model interactions among distant points, at the cost of interpretability, higher variance, and far greater compute — a poor trade when the signal is weak. We chose this trio deliberately: the simple and medium models answer "is there linear/interaction momentum in the features", and the complex model guards against the charge that a null is merely a feature-engineering failure.

Alternatives considered. A self-exciting point process (e.g. a Neural Hawkes model) is the most natural formal model of momentum, but its event-stream formulation does not map cleanly onto alternating-serve point data and would make the fair, shared-fold comparison harder; we keep it as future work. A plain LSTM was rejected as the complex model because it would not satisfy the not-taught-in-class requirement and offers no relational mechanism over the history. Pure "win-after-streak" hypothesis tests were rejected as the headline method because of the Miller–Sanjurjo bias; we retain them only as a sanity check whose bias the shuffle control absorbs.

5 Conclusion

We have framed tennis momentum as a falsifiable question — whether point *order* predicts the next point beyond serve, score and strength, and over what horizon — and built a from-scratch pipeline (temporal nested cross-validation, hand-implemented metrics, three models of increasing complexity) whose two controls make either answer defensible. State the finding once numbers are in: e.g. "momentum contributes a small but statistically detectable gain that saturates within k points", or "no predictive momentum is detectable above the shuffle null, despite the pipeline resolving planted effects as small as δ". The methodological contribution — controlling for the serve-and-strength confound and calibrating detectability — is what separates a real answer from the spurious streak effects that the literature warns against.

References

[1] Sackmann, J. (2024). *Grand Slam Point-by-Point Data*. GitHub. `https://github.com/JeffSackmann/tennis_slam_pointbypoint`

[2] Klaassen, F. J. G. M., & Magnus, J. R. (2001). Are points in tennis independent and identically distributed? Evidence from a dynamic binary panel data model. *Journal of the American Statistical Association*, 96(454), 500–509.

[3] Gilovich, T., Vallone, R., & Tversky, A. (1985). The hot hand in basketball: On the misperception of random sequences. *Cognitive Psychology*, 17(3), 295–314.

[4] Miller, J. B., & Sanjurjo, A. (2018). Surprised by the hot hand fallacy? A truth in the law of small numbers. *Econometrica*, 86(6), 2019–2047.

[5] Santoro, A., Faulkner, R., Raposo, D., Rae, J., Chrzanowski, M., Weber, T., Wierstra, D., Vinyals, O., Pascanu, R., & Lillicrap, T. (2018). Relational recurrent neural networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 31.